

WEEK-8

Session-15,16

TASK-1:

TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <fcntl.h>

#include <unistd.h>

#include <sys/wait.h>

#define INPUT_SIZE 256

int main(void)
{
    char input[INPUT_SIZE];
    char command[INPUT_SIZE];
    char filename[INPUT_SIZE];
    printf("Enter command with &> : ");
    fflush(stdout);
    if (fgets(input, sizeof(input), stdin) == NULL)
        return EXIT_FAILURE;
    input[strcspn(input, "\n")] = '\0';
    char *redirect = strstr(input, "&>");
    if (redirect == NULL)
    {
        printf("Error: '&>' not found.\n");
        return EXIT_FAILURE;
    }
}
```

```

size_t command_length = (size_t)(redirect - input);
if (command_length >= sizeof(command))
    command_length = sizeof(command) - 1;
strncpy(command, input, command_length);
command[command_length] = '\0';
int len = (int)strlen(command);
while (len > 0 &&
        (command[len - 1] == ' ' || command[len - 1] == '\t'))
{
    command[len - 1] = '\0';
    len--;
}
strncpy(filename, redirect + 2, sizeof(filename) - 1);
filename[sizeof(filename) - 1] = '\0';
char *file_start = filename;
while (*file_start == ' ' || *file_start == '\t')
    file_start++;
if (*file_start == '\0')
{
    printf("Error: Output filename is missing.\n");
    return EXIT_FAILURE;
}
printf("Command : %s\n", command);
printf("Output : %s\n", file_start);
int fd = open(file_start,
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

if (fd == -1)

```

```
{  
    perror("open");  
    return EXIT_FAILURE;  
}  
  
int saved_stdout = dup(STDOUT_FILENO);  
int saved_stderr = dup(STDERR_FILENO);  
if (saved_stdout == -1 || saved_stderr == -1)  
{  
    perror("dup");  
    close(fd);  
    return EXIT_FAILURE;  
}  
  
if (dup2(fd, STDOUT_FILENO) == -1)  
{  
    perror("dup2 stdout");  
    return EXIT_FAILURE;  
}  
  
if (dup2(fd, STDERR_FILENO) == -1)  
{  
    perror("dup2 stderr");  
    return EXIT_FAILURE;  
}  
  
close(fd);  
  
int status = system(command);  
  
dup2(saved_stdout, STDOUT_FILENO);  
dup2(saved_stderr, STDERR_FILENO);  
close(saved_stdout);  
close(saved_stderr);
```

```

if (status == -1)
{
    perror("system");
    return EXIT_FAILURE;
}

printf("Combined redirection completed successfully.\n");
printf("Both stdout and stderr were saved to: %s\n", file_start);

if (WIFEXITED(status))

    printf("Command exit status: %d\n", WEXITSTATUS(status));

return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~# nano combined_redirect.c
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g combined_redirect.c -o combined_redirect
root@LAPTOP-RJFD8EBT:~# ./combined_redirect
Enter command with &> : echo "Hello Output" ; ls /nonexistent &> combined.txt
Command : echo "Hello Output" ; ls /nonexistent
Output : combined.txt
Combined redirection completed successfully.
Both stdout and stderr were saved to: combined.txt
Command exit status: 2
root@LAPTOP-RJFD8EBT:~# cat combined.txt
Hello Output
ls: cannot access '/nonexistent': No such file or directory
root@LAPTOP-RJFD8EBT:~# ./combined_redirect
Enter command with &> : echo "START" ; ls /nonexistent ; echo "END" &> order.txt
Command : echo "START" ; ls /nonexistent ; echo "END"
Output : order.txt
Combined redirection completed successfully.
Both stdout and stderr were saved to: order.txt
Command exit status: 0
root@LAPTOP-RJFD8EBT:~# cat order.txt
START
ls: cannot access '/nonexistent': No such file or directory
END
root@LAPTOP-RJFD8EBT:~# ./combined_redirect
Enter command with &> : echo Hello
Error: '&>' not found.
root@LAPTOP-RJFD8EBT:~# ls -l combined_redirect.c combined_redirect combined.txt order.txt
-rw-r--r-- 1 root root 73 Sep 18 09:44 combined.txt
-rwxr-xr-x 1 root root 20176 Sep 18 09:31 combined_redirect
-rw-r--r-- 1 root root 2939 Sep 18 09:30 combined_redirect.c
-rw-r--r-- 1 root root 70 Sep 18 09:44 order.txt
root@LAPTOP-RJFD8EBT:~#

```

TASK-2:

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

CODE:

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

```

```

#define SIZE 512

#define MAX_STAGES 20

void trim(char *str)
{
    char *start = str;
    while (*start == ' ' || *start == '\t')
        start++;
    memmove(str, start, strlen(start) + 1);
    int len = strlen(str);
    while (len > 0 &&
        (str[len - 1] == ' ' || str[len - 1] == '\t'))
    {
        str[len - 1] = '\0';
        len--;
    }
}

int validate_command(char *input)
{
    if (strlen(input) == 0)
    {
        printf("Syntax Error: Empty command.\n");
        return 0;
    }

    if (strstr(input, "||") != NULL)
    {
        printf("Syntax Error: Invalid pipe usage.\n");
        return 0;
    }
}

```

```

if (strstr(input, "| >") != NULL ||
    strstr(input, "| >>") != NULL)
{
    printf("Syntax Error: Missing command after pipe.\n");
    return 0;
}
if (input[strlen(input) - 1] == '|')
{
    printf("Syntax Error: Command cannot end with pipe.\n");
    return 0;
}
if (strstr(input, ">") != NULL)
{
    char *p = strstr(input, ">");
    if (*(p + 1) == '\0' ||
        *(p + 1) == ' ' && *(p + 2) == '\0')
    {
        printf("Syntax Error: Missing output file.\n");
        return 0;
    }
}
printf("Syntax validation: VALID\n");
return 1;
}

void generate_plan(char *input)
{
    char copy[SIZE];
    strcpy(copy, input);

```

```

char *stages[MAX_STAGES];

int count = 0;

char *token = strtok(copy, "|");

while (token != NULL && count < MAX_STAGES)
{
    stages[count++] = token;
    token = strtok(NULL, "|");
}

printf("\n===== EXECUTION PLAN =====\n");

for (int i = 0; i < count; i++)
{
    trim(stages[i]);

    printf("Stage %d: %s\n", i + 1, stages[i]);

    if (i < count - 1)
        printf("    Output -> Pipe -> Stage %d\n", i + 2);
}

printf("\nExecution order:\n");

for (int i = 0; i < count; i++)
{
    printf("%d. %s\n", i + 1, stages[i]);
}

printf("=====\n");
}

int main(void)
{
    char input[SIZE];

    printf("=====\n");

```

```

printf("  COMPLEX COMMAND EXECUTOR  \n");
printf("=====\n");
printf("\nSupported:\n");
printf(" | Pipe\n");
printf(" <  Input redirection\n");
printf(" >  Output redirection\n");
printf(" >> Append redirection\n");
printf(" 2> Error redirection\n");
printf(" &> Combined redirection\n\n");
printf("Enter complex command: ");
if (fgets(input, sizeof(input), stdin) == NULL)
    return EXIT_FAILURE;
input[strcspn(input, "\n")] = '\0';
if (!validate_command(input))
    return EXIT_FAILURE;
generate_plan(input);
printf("\nExecuting command...\n\n");
int status = system(input);
if (status == -1)
{
    perror("system");
    return EXIT_FAILURE;
}
printf("\nCommand execution completed.\n");
return 0;
}

```

```

root@LAPTOP-RJFDBEST:~# nano complex_command.c
root@LAPTOP-RJFDBEST:~# gcc -Wall -Wextra -g complex_command.c -o complex_command
root@LAPTOP-RJFDBEST:~# echo "This is line one" > input.txt
root@LAPTOP-RJFDBEST:~# echo "This is line two" >> input.txt
root@LAPTOP-RJFDBEST:~# echo "Another line" >> input.txt
root@LAPTOP-RJFDBEST:~# ./complex_command
=====
Supported:
| Pipe
< Input redirection
> Output redirection
>> Append redirection
2> Error redirection
&> Combined redirection
Enter complex command: cat result.txt
Syntax validation: VALID
===== EXECUTION PLAN =====
Stage 1: cat result.txt
Execution order:
1. cat result.txt
=====
Executing command...
cat: result.txt: No such file or directory
Command execution completed.
root@LAPTOP-RJFDBEST:~# ./complex_command
=====
Supported:
| Pipe
< Input redirection
> Output redirection
>> Append redirection
2> Error redirection
&> Combined redirection
Enter complex command: cat result2.txt
Syntax validation: VALID
===== EXECUTION PLAN =====
Stage 1: cat result2.txt
Execution order:
1. cat result2.txt
=====
Executing command...
cat: result2.txt: No such file or directory
Command execution completed.
root@LAPTOP-RJFDBEST:~# ./complex_command
=====
Supported:
| Pipe
< Input redirection
> Output redirection
>> Append redirection
2> Error redirection
&> Combined redirection
Enter complex command: cat input.txt | grep line | wc -l
Syntax validation: VALID
===== EXECUTION PLAN =====
Stage 1: cat input.txt
Output -> Pipe -> Stage 2
Stage 2: grep line
Output -> Pipe -> Stage 3
Stage 3: wc -l
=====
Supported:
| Pipe
< Input redirection
> Output redirection
>> Append redirection
2> Error redirection
&> Combined redirection
Enter complex command: cat error.txt
Syntax validation: VALID
===== EXECUTION PLAN =====
Stage 1: cat error.txt
Execution order:
1. cat error.txt
=====
Executing command...
ls: cannot access '/nonexistent': No such file or directory
Command execution completed.
root@LAPTOP-RJFDBEST:~# ./complex_command
=====
Supported:
| Pipe
< Input redirection
> Output redirection
>> Append redirection
2> Error redirection
&> Combined redirection
Enter complex command: cat input.txt | grep line > result.txt
Syntax validation: VALID
===== EXECUTION PLAN =====
Stage 1: cat input.txt
Output -> Pipe -> Stage 2
Stage 2: grep line > result.txt
Execution order:
1. cat input.txt
2. grep line > result.txt
=====
Executing command...
Command execution completed.
root@LAPTOP-RJFDBEST:~# ^C
root@LAPTOP-RJFDBEST:~#

```

TASK-3:

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

Code:

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/wait.h>

#include <signal.h>

#define MAX_JOBS 10

#define INPUT_SIZE 256

typedef struct
{
    int id;

    pid_t pid;

    char command[INPUT_SIZE];

    int running;
} Job;

```

```

Job jobs[MAX_JOBS];

int job_count = 0;

int next_job_id = 1;


void check_jobs(void)
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            int status;

            pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

            if (result == jobs[i].pid)
            {
                jobs[i].running = 0;

                printf("\n[%d] Done   %s\n",
                    jobs[i].id, jobs[i].command);
            }
        }
    }
}

void show_jobs(void)
{
    check_jobs();

    if (job_count == 0)
    {
        printf("No jobs.\n");

        return;
    }
}

```

```

    }

    printf("\nJob List:\n");
    for (int i = 0; i < job_count; i++){
        printf("[%d] PID=%d %s %s\n",
            jobs[i].id,
            jobs[i].pid,
            jobs[i].running ? "Running" : "Done",
            jobs[i].command);
    }
}

void start_background_job(char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Maximum job limit reached.\n");
        return;
    }

    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        execl("/bin/sh", "sh", "-c", command, (char *)NULL);
        perror("exec");
        exit(EXIT_FAILURE);
    }
}

```

```

jobs[job_count].id = next_job_id++;
jobs[job_count].pid = pid;
snprintf(jobs[job_count].command,
        INPUT_SIZE,
        "%s",
        command);
jobs[job_count].running = 1;
printf("[%d] %d started in background\n",
        jobs[job_count].id, pid);
job_count++;
}
int main(void)
{
    char input[INPUT_SIZE];

    printf("=== Background Job Manager ===\n");
    printf("Use '&' for background jobs.\n");
    printf("Commands: jobs, exit\n\n");

    while (1)
    {
        check_jobs();

        printf("bg-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
    }
}

```

```
input[strcspn(input, "\n")] = '\0';
```

```
if (strlen(input) == 0)
```

```
    continue;
```

```
if (strcmp(input, "exit") == 0)
```

```
    break;
```

```
if (strcmp(input, "jobs") == 0)
```

```
{
```

```
    show_jobs();
```

```
    continue;
```

```
}
```

```
int len = strlen(input);
```

```
if (input[len - 1] == '&')
```

```
{
```

```
    input[len - 1] = '\0';
```

```
while (strlen(input) > 0 &&
```

```
    input[strlen(input) - 1] == ' ')
```

```
{
```

```
    input[strlen(input) - 1] = '\0';
```

```
}
```

```
if (strlen(input) == 0)
```

```
{
```

```
    printf("Error: Empty background command.\n");
```

```

        continue;
    }

    start_background_job(input);
}

else
{
    printf("Use '&' at the end for a background job.\n");
}
}

/* Clean up remaining jobs */
for (int i = 0; i < job_count; i++)
{
    if (jobs[i].running)
        waitpid(jobs[i].pid, NULL, 0);
}

printf("Exiting background job manager.\n");

return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~# nano background_jobs.c
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g background_jobs.c -o background_jobs
root@LAPTOP-RJFD8EBT:~# ./background_jobs
=== Background Job Manager ===
Use '&' for background jobs.
Commands: jobs, exit

bg-shell> sleep 5 &
[1] 7602 started in background
bg-shell> jobs

[1] Done      sleep 5

Job List:
[1] PID=7602 Done sleep 5
bg-shell> sleep 3 &
[2] 7606 started in background
bg-shell> jobs

[2] Done      sleep 3

Job List:
[1] PID=7602 Done sleep 5
[2] PID=7606 Done sleep 3
bg-shell> exit
Exiting background job manager.
root@LAPTOP-RJFD8EBT:~# |

```

TASK-4:

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/wait.h>

#include <signal.h>


#define MAX_JOBS 10

#define CMD_SIZE 100

typedef enum

{

    RUNNING,

    STOPPED,

    COMPLETED

} JobState;

typedef struct

{

    int job_id;

    pid_t pid;

    pid_t pgid;

    char command[CMD_SIZE];

    JobState state;

} Job;

Job job_table[MAX_JOBS];

int job_count = 0;

int next_job_id = 1;
```

```

const char *state_name(JobState state)
{
    switch (state)
    {
        case RUNNING:
            return "Running";
        case STOPPED:
            return "Stopped";
        case COMPLETED:
            return "Completed";
        default:
            return "Unknown";
    }
}

void add_job(pid_t pid, pid_t pgid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table full.\n");
        return;
    }
    job_table[job_count].job_id = next_job_id++;
    job_table[job_count].pid = pid;
    job_table[job_count].pgid = pgid;
    job_table[job_count].state = RUNNING;
    snprintf(job_table[job_count].command,
             CMD_SIZE, "%s", command);
    job_count++;
    printf("[%d] PID=%d PGID=%d Running\n",

```

```

        job_table[job_count - 1].job_id,
        pid,
        pgid);
}

void update_jobs(void)
{
    for (int i = 0; i < job_count; i++)
    {
        if (job_table[i].state == RUNNING)
        {
            int status;

            pid_t result = waitpid(job_table[i].pid,
                                   &status,
                                   WNOHANG);

            if (result == job_table[i].pid)
            {
                job_table[i].state = COMPLETED;
            }
        }
    }
}

void remove_completed_jobs(void)
{
    for (int i = 0; i < job_count;)
    {
        if (job_table[i].state == COMPLETED)
        {
            printf("Removing completed job [%d]\n",
                   job_table[i].job_id);

```

```

        for (int j = i; j < job_count - 1; j++)
        {
            job_table[j] = job_table[j + 1];
        }
        job_count--;
    }
    else
    {
        i++;
    }
}

void show_jobs(void)
{
    update_jobs();

    if (job_count == 0)
    {
        printf("Job table is empty.\n");
        return;
    }
    printf("\n===== JOB TABLE =====\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d PGID=%d State=%s Command=%s\n",
            job_table[i].job_id,
            job_table[i].pid,

```

```

        job_table[i].pgid,
        state_name(job_table[i].state),
        job_table[i].command);
    }
    printf("=====\n");
}

void validate_jobs(void)
{
    int valid = 1;

    for (int i = 0; i < job_count; i++)
    {
        if (job_table[i].pid <= 0 ||
            job_table[i].pgid <= 0 ||
            strlen(job_table[i].command) == 0)
        {
            valid = 0;
            break;
        }
    }

    if (valid)
        printf("Job records: VALID\n");
    else
        printf("Job records: INVALID\n");
}

void start_job(char *command)
{
    pid_t pid = fork();

```

```

if (pid < 0)
{
    perror("fork");
    return;
}
if (pid == 0)
{
    setpgid(0, 0);
    execl("/bin/sh", "sh", "-c", command, (char *)NULL);
    perror("exec");
    exit(EXIT_FAILURE);
}
setpgid(pid, pid);
add_job(pid, pid, command);
}

int main(void)
{
    char input[CMD_SIZE];
    printf("=== JOB TABLE MANAGER ===\n");
    printf("Commands:\n");
    printf(" command & - start background job\n");
    printf(" jobs    - display job table\n");
    printf(" validate - validate job records\n");
    printf(" remove   - remove completed jobs\n");
    printf(" exit     - exit\n\n");
    while (1)
    {
        update_jobs();
        printf("job-shell> ");
    }
}

```

```
fflush(stdout);
if (fgets(input, sizeof(input), stdin) == NULL)
    break;
input[strcspn(input, "\n")] = '\0';
if (strcmp(input, "exit") == 0)
    break;
if (strcmp(input, "jobs") == 0)
{
    show_jobs();
}
else if (strcmp(input, "validate") == 0)
{
    validate_jobs();
}
else if (strcmp(input, "remove") == 0)
{
    update_jobs();
    remove_completed_jobs();
}
else
{
    int len = strlen(input);

    if (len > 0 && input[len - 1] == '&')
    {
        input[len - 1] = '\0';

        while (strlen(input) > 0 &&
            input[strlen(input) - 1] == ' ')
```

```

    {

        input[strlen(input) - 1] = '\0';

    }

    start_job(input);

} else {

    printf("Use '&' for a background job.\n");

}

}

for (int i = 0; i < job_count; i++)

{

    if (job_table[i].state == RUNNING)

        waitpid(job_table[i].pid, NULL, 0);

}

printf("Job table manager exited.\n");

return 0;

}

```

```

root@LAPTOP-RJFDBEST:~# nano job_table.c
root@LAPTOP-RJFDBEST:~# gcc -Wall -Wextra -g job_table.c -o job_table
root@LAPTOP-RJFDBEST:~# ./job_table
=== JOB TABLE MANAGER ===
Commands:
  command & - start background job
  jobs      - display job table
  validate  - validate job records
  remove    - remove completed jobs
  exit      - exit

job-shell> sleep 5 &
jobs
validate[1] PID=7626 PGID=7626 Running
job-shell>
===== JOB TABLE =====
[1] PID=7626 PGID=7626 State=Running Command=sleep 5
=====
job-shell> sleep 5 &
[2] PID=7631 PGID=7631 Running
job-shell> sh: 1: validatesleep: not found
^C
root@LAPTOP-RJFDBEST:~# nano job_table.c
root@LAPTOP-RJFDBEST:~# gcc -Wall -Wextra -g job_table.c -o job_table
root@LAPTOP-RJFDBEST:~# ./job_table
=== JOB TABLE MANAGER ===
Commands:
  command & - start background job
  jobs      - display job table
  validate  - validate job records
  remove    - remove completed jobs
  exit      - exit

job-shell> sleep 5 &
[1] PID=7641 PGID=7641 Running
job-shell> jobs
===== JOB TABLE =====
[1] PID=7641 PGID=7641 State=Completed Command=sleep 5
=====
job-shell> validate
Job records: VALID
job-shell> jobs
===== JOB TABLE =====
[1] PID=7641 PGID=7641 State=Completed Command=sleep 5
=====
job-shell> remove
Removing completed job [1]
job-shell> jobs
Job table is empty.
job-shell> exit
Job table manager exited.
root@LAPTOP-RJFDBEST:~#

```

